

AMIRIS Project

Working Session Summary

Germany 2018 Backtest: Setup, Execution, and Validation

Prepared 2026-07-13

1. Project Overview

AMIRIS (Agent-based Market model for the Investigation of Renewable and Integrated energy Systems) is an agent-based electricity market simulator developed by the German Aerospace Center (DLR). Each market participant - power plant operators, traders, storage/flexibility units, demand, and the market itself - is modelled as an explicit software agent that bids into an hourly day-ahead market. The engine is a Java application (amiris-core_4.1.2-jar-with-dependencies.jar); scenarios are authored in YAML and compiled/executed/extracted using the Python tools fameio and amirisp.py.

Local project layout

- examples/backtest/<Country><Year>/ - historical scenario definitions (inputs)
- result/ - output of the most recently run simulation
- amiris-core_4.1.2-jar-with-dependencies.jar - the simulation engine
- py313env/ - Python virtual environment containing amirisp.py, fameio, pandas, matplotlib

2. Understanding the result/ Folder

Every AMIRIS run produces one output.pb (binary protobuf - the raw source of truth) plus a metadata.json describing every field, and one CSV per agent type, extracted from output.pb. Each CSV row is keyed by AgentId + TimeStep (hourly).

File	What it contains
DayAheadMarketSingleZone.csv	The market-clearing price (EUR/MWh) and awarded volume each hour - the core output
ConventionalPlantOperator.csv	Per-plant CO2 emissions, fuel use, revenue, costs
RenewableTrader.csv / NoSupportTrader.csv / SystemOperatorTrader.csv	Renewables marketed under different support schemes: potential vs. awarded energy, support payments
GenericFlexibilityTrader.csv	Storage dispatch: charge/discharge energy, price, state of charge
DemandTrader.csv	Demand-side bidding
SensitivityForecaster.csv	Internal price/volume forecasts used by agents for planning

Key clarification established early in this project: the timeseries/*.csv files bundled with each scenario (load, wind/solar profiles, fuel prices) are real historical inputs, not something AMIRIS regenerates. The only genuinely simulated/emergent output is market behaviour - dispatch and price - which is why DayAheadMarketSingleZone.csv is the one column validated against reality.

3. Understanding the backtest/ Folder

examples/backtest/ contains six historical scenarios: Austria2019 and Germany2015 through Germany2019. Each follows the same structure: scenario.yaml (simulation start/stop time, random seed, includes), schema.yaml (generated type schema), agents/*.yaml and contracts/*.yaml (who trades with

whom), and timeseries/*.csv (the real historical drivers).

What changes year to year

- Germany2015/2016: baseline setup with *_availability.csv files per fuel type
- Germany2017: adds backup_storage_dispatch.csv, specifically to capture the 'Kalte Dunkelflaute' event of January 2017 (a multi-day cold spell with almost no wind/solar output)
- Germany2018: same structure as 2016/2017
- Germany2019: most detailed - availability split into separate must-run and outage files per fuel type, plus 18 competing storage units

4. Running a Simulation

amirisp.py (already installed in py313env) wraps the full compile -> execute -> extract pipeline:

```
. \py313env\Scripts\Activate.ps1
amiris -l info run --scenario "examples\backtest\Germany2018\scenario.yaml" --jar
"amiris-core_4.1.2-jar-with-dependencies.jar" --output "result_Germany2018"
```

Gotcha encountered and fixed: --log/-l is a top-level option of the amiris command, not an option of the run subcommand. Placing it after 'run' fails with 'unrecognized arguments'; it must go before the subcommand, as shown above.

The Germany2018 run completed successfully, producing a full year (8,760 hourly rows) of output across all agent CSVs.

5. Getting Real Historical Prices for Validation

First attempt: ENTSO-E Transparency Platform

The user signed up and pulled a link, but had selected the wrong bidding zone (10YDOM-CZ-DE-SKK, a CZ-DE-SK capacity domain rather than Germany's actual price zone). Guidance given: use DE-LU for the zone from 1 Oct 2018 onward, and DE-AT-LU for before that date, since Germany and Austria shared one price zone until the market split on 1 October 2018. An entsoe-py based scripted-download approach was proposed as a more reliable alternative to the web UI's per-view export limits.

Actual approach used: SMARD.de (Bundesnetzagentur)

The user switched to SMARD's download centre instead. First attempt failed silently: the downloaded files' Start date/End date columns showed Jun-Jul 2026 (i.e. 'today'), not 2018, because the date-range picker was not explicitly applied before downloading - a known SMARD UI gotcha. Corrected by explicitly setting the date range before download.

The corrected pair of files:

- Day-ahead_prices_201801010000_201810010000_Hour.csv (Jan-Sep 2018) - real values live in the 'DE/AT/LU' column; 'Germany/Luxembourg' and 'Austria' columns are blank for this period
- Day-ahead_prices_201810010000_201901010000_Hour.csv (Oct-Dec 2018) - real values live in the 'Germany/Luxembourg' column; 'DE/AT/LU' is blank for this period

This confirmed the bidding-zone split exactly as expected: negative prices visible in the Jan 2018 DE/AT/LU data (e.g. -63.14 EUR/MWh on 1 Jan) matched known real-world New Year oversupply events.

6. Merging the Real Price Data

`merge_smard_prices.py` takes the DE/AT/LU column from the pre-split file and the Germany/Luxembourg column from the post-split file, concatenates them into one continuous hourly series, and writes `germany_2018_real_prices.csv` (columns: Timestamp, price_eur_mwh).

Check	Result
Pre-split rows (Jan-Sep)	6,551
Post-split rows (Oct-Dec)	2,209
Merged total	8,760 (= a full non-leap year)
Duplicate / missing timestamp	1, at 2018-03-25 02:00 (spring DST transition)

The single DST artifact hour is expected whenever hourly local-time data spans a clock-change date and is negligible (1 of 8,760 hours, ~0.01%).

7. Comparing Simulated vs. Real Prices

`compare_price.py` joins the simulated `ElectricityPriceInEURperMWh` series against the merged real-price series on timestamp, and reports MAE, RMSE, and correlation. A defensive fix was added to drop duplicate-timestamp rows on the real side (the DST artifact from step 6), which initially caused a 'cannot reindex on an axis with duplicate labels' error.

Full-year and split-period results

Period	Hours	MAE (EUR/MWh)	RMSE (EUR/MWh)	Correlation
Full year 2018	8,759	8.84	11.72	0.773
Before 1 Oct (joint DE/AT/LU price)	6,551	9.04	12.08	0.747
On/after 1 Oct (true DE-only price)	2,208	8.25	10.57	0.847

Interpretation: 2018's average day-ahead price was roughly in the low 40s EUR/MWh, so an MAE of ~9 EUR/MWh is meaningful (~20% relative error) but not alarming for a backtest. Correlation improves markedly (0.747 → 0.847) once the comparison is like-for-like after the October zone split, confirming that part of the pre-split mismatch was a genuine structural artefact: the Germany2018 scenario simulates Germany as a standalone zone even for the period when the real market cleared jointly with Austria. MAE improves only modestly across the split, indicating a second, independent source of error beyond the zone-coupling effect.

Chart

price_comparison.png overlays both series across the year. They track closely in level and seasonal shape (both dip in shoulder months, both rise into winter, both show matching negative-price troughs around New Year and other oversupply periods). The main visible divergence is damped spike magnitude: the simulated series tracks the timing of real price spikes but frequently undershoots their peak height - a common AMIRIS backtest signature pointing to bidding/forecast behaviour smoothing out scarcity pricing.

8. Largest-Error Hour Analysis

The 25 largest absolute-error hours in price_comparison.csv cluster into three distinct, non-random patterns:

Pattern A: Public holidays - model floors at zero instead of going negative

1 Jan 2018 (02:00-12:00) and 1 May 2018 (14:00-15:00): simulated price sits at a flat 0.00 for every one of these hours while real price falls to between -49 and -76 EUR/MWh. Both dates are German public holidays with low industrial demand and high renewable output, which in reality pushes prices deeply negative because subsidised renewable operators keep bidding to protect feed-in-tariff eligibility, and conventional plants pay to avoid costly shutdown/restart. The model's flat zero suggests its renewable/must-run bidding logic does not allow negative bids.

Pattern B: Feb 5-19, 2018 - model overshoots during a cold-weather period

Simulated prices of 92-103 EUR/MWh against real prices of 45-53 EUR/MWh across several days in this window, coinciding with a European cold spell in February 2018. Root cause confirmed by direct inspection - see Section 10.

Pattern C: 9 Oct 2018, 19:00 - isolated evening-peak undershoot

Simulated 61.85 vs. real 110.28 EUR/MWh - a single hour, consistent with the general 'damped spike' behaviour noted in the chart rather than a systematic multi-day pattern.

9. Root Cause Analysis: Jan 1, 2018 Price Flooring

Pulled AgentId-level rows from RenewableTrader.csv, NoSupportTrader.csv, SystemOperatorTrader.csv and ConventionalPlantOperator.csv for 2018-01-01 00:00-14:00, and cross-referenced against the agent definitions in Germany2018/agents/Conventionals.yaml and RenewablesAndPolicy.yaml.

Agent	Observed behaviour, 00:00-14:00
SystemOperatorTrader (FIT)	Awarded = Offered = Potential every hour - 100% must-take
RenewableTrader / NoSupportTrader	Offered > Awarded in early hours (e.g. 00:00: offered 19,370 MWh, awarded 17,213 MWh) - genuine curtailment, gap closes by ~07:00
Nuclear (Agent 500)	Flat 9,423 MWh awarded every hour - full baseload commitment
Lignite (Agent 501)	Flat 14,007 of 18,310 MWh offered, every hour - constant, price-insensitive floor

Agent	Observed behaviour, 00:00-14:00
HardCoal / GasCCGT / GasOCGT / Oil	Zero awarded all 15 hours - fully undercut

The diagnostic signal is that RenewableTrader/NoSupportTrader are genuinely curtailed (offered > awarded) at the exact same hours the clearing price sits at a flat 0.00 - real curtailment with no negative price is the signature of a zero-cost price floor. RenewablesAndPolicy.yaml sets OpexVarInEURperMWH: 0.0 for these VariableRenewableOperator agents, so whenever one of them is the marginal (last-accepted) bid, the market clears at 0, not below it. Nuclear and lignite traders do have negative bid bands defined in Conventionals.yaml (minMarkup/maxMarkup of -150/-100 and -80/-10 respectively), but their dispatch in this window is flat and inelastic rather than actually probing negative territory.

Confirmed root cause: the model's renewable/no-support trader agents bid at exactly their zero variable cost rather than a negative price, so the simulation cannot reproduce real negative-price events even on hours where it does curtail renewable output. Real German prices fell to -76 EUR/MWh on this date because real subsidised renewables and must-run contracts bid negative to protect their premiums - a behaviour this scenario's renewable agents do not reproduce.

10. Root Cause Analysis: Feb 5-19, 2018 Overshoot

Computed daily mean capacity factors (AwardedEnergyInMWH / InstalledPowerInMW) per fuel type across the window, then checked the actual timeseries input files against their full-year average to test whether a capacity shortage or a cost-input spike was responsible.

Input	Annual avg	Feb 5-19 avg	Ratio	Direction
CO2 price (EUR/t)	19.57	9.23	0.47	Lower (not a cost driver)
Hard coal price	13.23	12.38	0.94	~Normal
Natural gas price	20.24	18.80	0.93	~Normal
Hard coal outage factor	0.70	0.31	0.44	Less unavailable (more capacity)
Lignite outage factor	0.59	0.14	0.23	Less unavailable (more capacity)
Load (demand, MWh/h)	68,400	77,999	1.14	+14% higher demand

Coal and gas capacity factors during this window ran at 0.5-0.97 - moderate to high, but not saturated at 1.0 - ruling out a pure capacity-shortage explanation. None of the cost or outage inputs point toward higher prices: CO2 was cheaper than average and plant availability was better than average. The one genuine anomaly is demand running about 14% above the annual average, consistent with a real February 2018 cold snap.

Confirmed root cause: these backtest scenarios model Germany as an isolated single zone with no cross-border import/export agents. In reality, a demand spike of this size is partly absorbed by cheaper imports from neighbouring grids; the model has no such relief valve, so it must climb further up its own domestic merit order - into the steeper gas-peaker bid bands (GasOCGT markup range 15-60 EUR/MWh in Conventionals.yaml) - to clear the same demand, overshooting the real, import-cushioned price.

Together, Sections 9 and 10 identify two distinct, independent model limitations rather than one bug: (1) renewable/no-support traders cannot bid negative, so negative-price events are never reproduced, and (2) the absence of cross-border trade modelling means domestic price responds more steeply to demand spikes than the real interconnected market did.

11. Extending the Backtest to All Years (2015-2019)

The full run -> extract real prices -> compare workflow was repeated for Germany2015, 2016, 2017 and 2019 (Germany2018 already covered in Sections 7-10), using the same amirispys run command and the same SMARD.de source for real prices. None of these four years cross the DE/AT/LU to DE-LU zone split, so each needed only a single column extracted from a single SMARD file (extract_smard_prices.py), rather than the two-file merge used for 2018.

Year	Hours compared	MAE (EUR/MWh)	RMSE (EUR/MWh)	Correlation
2015	8,663	6.64	8.67	0.774
2016	8,759	4.96	6.92	0.833
2017	8,759	5.70	8.76	0.871
2018	8,759	8.84	11.72	0.773
2019	8,759	5.37	8.01	0.857

Note: the 2015 real-price export from SMARD only has data from 5 Jan 2015 onward (the first 4 days show no value in the DE/AT/LU column in SMARD's own export, not an artefact introduced during extraction), so its row count and statistics are not perfectly comparable to the other four years.

Cross-year findings

- 2018 is clearly the worst-fitting year (highest MAE and RMSE, tied-lowest correlation) - consistent with it being the one year straddling the zone split, and the one year with the two confirmed structural gaps documented in Sections 9 and 10.
- 2016 and 2017 fit best (MAE under 6 EUR/MWh, correlation 0.83-0.87) - both are clean, single-zone, no-split years, supporting the theory that removing the zone-split mismatch meaningfully improves the fit.
- 2019 fits nearly as well as 2016/2017 (MAE 5.37, correlation 0.857) despite being the most structurally complex scenario (18 competing storage units, split must-run/outage inputs) - the added modelling detail does not appear to hurt the price fit.
- 2015 has the weakest correlation of the non-2018 years (0.774) despite a middling MAE - worth revisiting once/if a complete (Jan 1-4 inclusive) real-price series becomes available.

12. Recommendations for Improving the Model Fit

- To reproduce negative-price events: allow RenewableTrader/NoSupportTrader agents to bid below their zero variable cost (a configurable negative markup, mirroring how Nuclear/Lignite traders already do it in Conventionals.yaml) on hours where curtailment is expected.
- To reduce the Feb-type overshoot: introduce a cross-border import/export mechanism (even a

simplified net-exchange timeseries subtracted from/added to load) so domestic price does not have to absorb the full magnitude of demand spikes alone.

- For any pre-October 2018 backtest, treat the comparison as inherently approximate given the DE/AT/LU vs. DE-only structural mismatch; consider it a secondary, not primary, validation period - confirmed by Section 11's cross-year table, where 2018 is the clear outlier.
- Consider digging into 2015's weaker correlation the same way Sections 9-10 dug into 2018, to check whether it shares either of the two confirmed 2018 root causes or has a distinct explanation of its own.

13. Practical Command Reference

Run a backtest scenario

```
amiris -l info run --scenario "examples\backtest\Germany<YEAR>\scenario.yaml" --jar "amiris-core_4.1.2-jar-with-dependencies.jar" --output "result_Germany<YEAR>"
```

Merge SMARD pre/post zone-split price exports

```
python merge_smard_prices.py --pre-split <jan_sep.csv> --post-split <oct_dec.csv> --output germany_<year>_real_prices.csv
```

Compare simulated vs. real price, with optional zone-split reporting

```
python compare_price.py --sim result_Germany<YEAR>\DayAheadMarketSingleZone.csv --real germany_<year>_real_prices.csv --split-date <YYYY-MM-DD>
```

14. Phase 2: Extending to 2023 via the ENTSO-E API

With the supervisor's confirmation of the Brainpool comparison year still pending, work began on 2023 in parallel, since 2023 is very likely to be needed regardless (either as the benchmarking year itself, or as useful additional backtesting evidence). Unlike 2015-2019, AMIRIS does not ship a ready-made Germany2023 scenario, so every input had to be sourced and built from scratch.

Why 2023 (and not 2020-2022)

2020-2021 carry a COVID-19 demand shock, and 2021-2022 carry the gas-crisis price distortion - both make for messy, hard-to-interpret backtests. 2023 is not perfectly clean either (Germany's last three nuclear plants shut down permanently in April 2023, and gas/CO2 prices were still elevated post-crisis), but it is a reasonable, well-motivated next step, and the one most likely to overlap with the department's purchased Brainpool data.

Getting API access

An ENTSO-E Transparency Platform account already existed from the earlier price-download work, but the RESTful API token option was not visible under that account's settings. A request was drafted and sent to transparency@entsoe.eu asking for API access to be enabled, referencing the thesis context. The token was subsequently obtained and verified with a live test query before any further work was built on top of it.

Data pulled so far

Using the confirmed ENTSO-E token, four things were pulled for the German/Luxembourg bidding zone: installed generation capacity per technology (2023 snapshot), hourly electricity demand for the full year, hourly generation for each renewable technology (solar, wind onshore/offshore, run-of-river, biomass, other renewables - converted into AMIRIS's normalised 0-1 profile format), and conventional-plant outage events for nuclear, lignite and hard coal (converted into an hourly unavailability-fraction series). The reusable scripts (`fetch_entsoe_2023_data.py` and `fetch_2023_outages.py`) are kept in the project folder. Fuel prices (oil, gas, hard coal) were pulled separately from FRED, a different free public source, since ENTSO-E does not carry commodity prices - see below.

Technology	2019 scenario (MW)	2023 (ENTSO-E, MW)	Change
Nuclear	9,516	4,056	-57%
Lignite	21,275	17,692	-17%
Hard coal	25,633	18,559	-28%
Natural gas	~26,525	31,878	+20%
Oil	4,084	4,083	~0%
Solar	~44,000	63,366	+44%
Wind onshore	~54,674	57,741	+6%
Wind offshore	~5,051	8,129	+61%

These changes all match well-known, independently verifiable German energy-transition trends (ongoing coal/nuclear phase-out, continued solar and offshore wind growth), which is a good consistency check on the data pulled.

Handling the mid-year nuclear phase-out

ENTSO-E's installed-capacity figure is a single annual snapshot (taken at the start of the year), so it reflects capacity before the April 2023 shutdown, not zero for the whole year. Rather than splitting 2023 into two separate sub-scenarios, the pre-shutdown capacity was kept as the year's `InstalledPowerInMW` value, and an hourly nuclear outage-factor file was built that is 0% unavailable before the shutdown date and 100% unavailable after - reusing a mechanism (hourly outage factors) that AMIRIS's existing scenarios already rely on for other fuel types, rather than inventing a new one. This has since been implemented (see below).

Step-by-step: the full 2023 data-gathering procedure actually followed

- 1. Verify API access: a short live query (day-ahead prices for a single week) confirmed the ENTSO-E token was active before anything else was built on top of it.
- 2. Pull installed capacity per technology: one call to `query_installed_generation_capacity` gave a single 2023 snapshot value per fuel/technology, used both to configure AMIRIS's `InstalledPowerInMW` attributes and to normalise renewable generation into 0-1 profiles.
- 3. Pull hourly load and renewable generation: `query_load` and `query_generation` (filtered per technology by ENTSO-E's production-type codes) gave the full year at 15-minute resolution; downsampled to hourly, a single daylight-saving data gap was interpolated, and each renewable

series was divided by its installed capacity to produce the normalised profile AMIRIS expects.

- 4. Pull conventional plant outages: `query_unavailability_of_generation_units` gave individual outage events (start/end time, capacity affected) for nuclear, lignite and hard coal, which were cleaned (cancelled reports and superseded revisions removed) and converted into an hourly 'fraction unavailable' series per fuel type using a sweep-line aggregation. The nuclear series was then manually overridden to 100% unavailable from 15 April 2023 onward, since permanent plant closures are not represented in this outage-event data at all (see obstacle log, next section).
- 5. Pull fuel prices: Brent oil (daily), and IMF/World-Bank-benchmark European gas and Australian coal prices (monthly) were pulled from FRED - a free, public source - then converted from their native units (USD/barrel, USD/MMBtu, USD/tonne) into the EUR/MWh_th unit AMIRIS's existing scenarios use, using a daily EUR/USD exchange rate (also from FRED) and disclosed energy-content conversion factors.
- 6. CO2 price: no free bulk-download source was found for EU ETS allowance prices; this step was handed off to the student to source separately (e.g. university commercial data access, or a small manual pull of reference points), rather than guessing at a figure.
- 7. Still to do: assemble the actual Germany2023 `scenario.yaml/agents/contracts` files (by adapting Germany2019's structure with the 2023 capacities and prices gathered above), then run and validate the scenario.

Fuel price sourcing and conversion factors

File	Source	Native unit	2023 mean (EUR/MWh_th)
oil_price.csv	FRED: Brent spot (DCOILBRENTU), daily	USD/barrel	44.88
natural_gas_price.csv	FRED: IMF EU gas benchmark (PNGASEUUSDM), monthly	USD/MMBtu	40.69
hard_coal_price.csv	FRED: IMF Australia coal benchmark (PCOALAUUSDM), monthly	USD/tonne	24.95
co2_price.csv	Not yet sourced - handed off to student	-	-

All three converted series were sanity-checked against independently known 2023 market levels (e.g. gas falling from roughly 60 EUR/MWh in January to the low 30s by mid-year, matching the widely reported post-crisis gas price collapse) and found consistent. Three disclosed unit-conversion assumptions apply: gas uses the exact energy-unit conversion 1 MMBtu = 0.293071 MWh; coal assumes roughly 6.978 MWh per tonne (a ~6,000 kcal/kg net calorific value, typical for the benchmark thermal coal quality priced by this series); oil assumes roughly 1.7 MWh per barrel of Brent crude (a commonly used approximate figure). These are genuine methodological choices worth restating explicitly in the thesis text, not silently-applied constants.

15. Comprehensive Obstacles Log: Every Problem Encountered, and How It Was Solved

This section exists as a single reference point for every practical problem hit across the whole project so far, and exactly how each was resolved. Nothing here changed a conclusion after the fact - every issue

was caught and fixed before it affected any reported result.

#	Obstacle	How it was solved
1	Wrong ENTSO-E bidding zone selected initially (a CZ-DE-SK capacity domain code instead of Germany's actual price zone)	Identified the correct zone codes (DE-LU post-Oct-2018, DE-AT-LU before) and the exact 1 Oct 2018 split date
2	SMARD.de download silently defaulted to the current date instead of the requested historical year	Caught by checking the actual dates inside the downloaded file before use; fixed by explicitly re-applying the date range before downloading
3	2018 real-price data was split across two different bidding zones mid-year (Germany-Austria joint zone, then Germany-only)	Downloaded both halves separately and merged them into one continuous series at the exact split date
4	Comparison script crashed on a duplicate timestamp (a daylight-saving collision in the merged real-price data)	Dropped duplicate index entries before joining, keeping the first occurrence
5	Chart generation failed due to a broken Tcl/Tk graphics library on this machine	Switched matplotlib to its non-interactive 'Agg' backend, which only needs to save an image file, not display one
6	'amiris run --log info' failed with an 'unrecognized arguments' error	--log-l is a top-level option of the amiris command, not of the run subcommand; moved it before 'run' in the command
7	PDF generation crashed on special characters (a bullet symbol, and a deprecated core-font name)	Switched to plain hyphen bullets and the 'Helvetica' font name instead of 'Arial'
8	Duplicating a PowerPoint slide broke its embedded chart image (the picture reference did not carry over correctly)	Explicitly copied the underlying image data and rebuilt the picture's internal reference in the new slide
9	A new slide's chart image visually overlapped its own text once more bullet points were added than the original template had	Repositioned and resized the chart to leave clear vertical space below the (now longer) text block
10	Could not save an updated PDF/PPTX because the file was open in a viewer on the user's machine (Windows locks open files)	Generated the corrected version to a temporary location first, then copied it into place once the original was closed
11	Tables in the plain-language guide had overlapping text where long descriptions did not fit their column	Replaced manual fixed-height table cells with the PDF library's built-in wrapping table feature, which grows row height to fit content
12	ENTSO-E account had no visible option to generate an API token	Sent a direct request to ENTSO-E support (transparency@entsoe.eu) asking for API access to be enabled, explaining the academic/thesis context
13	2023 load and generation data came back at 15-minute resolution instead of the expected hourly resolution (4x too many rows)	Downsampled every series to hourly averages before use
14	A single hour of 2023 data was missing entirely, at the exact moment of the March daylight-saving clock change	Filled the single gap using linear interpolation between the surrounding hours - a real, well-known artefact of European grid data around clock-change dates, not a fetching error
15	Outage-event data contained non-numeric values in its capacity columns, crashing a subtraction calculation	Coerced both columns to numeric and dropped the small number of rows that still had no valid value
16	The first outage-data endpoint tried (query_unavailability_of_production_units) returned zero matching events for nuclear/lignite/hard coal	Discovered it covers a different registration category entirely (aggregated wind/storage assets, not individually-registered conventional plants); reverted to the correct endpoint (query_unavailability_of_generation_units), which had already been validated

#	Obstacle	How it was solved
17	That correct endpoint only reports 'Planned maintenance' events for Germany, with no forced/unplanned outages at all	Confirmed this is a genuine limitation of what Germany's TSOs publish there (not a filtering mistake); documented as a known caveat - 2023's coal/lignite outage factors likely understate true unavailability, which could bias simulated prices slightly downward for those years
18	No free, bulk-downloadable source found for EU ETS (CO2 allowance) historical prices	Every other 2023 input was still built and saved; CO2 price sourcing was handed off to the student (university data access, or a small manual reference-point pull) rather than guessing at values for a thesis-grade input

16. Project Direction Update: Same-Assumptions Comparison

Following a meeting with the supervisor, the benchmarking methodology (Comparison 2) was refined into a more rigorous design than originally planned.

What changes

The original plan (and the backtesting work in Sections 7-14) compares AMIRIS's output against real historical prices, using AMIRIS's own independently-sourced input data. That design cannot distinguish two different explanations for any mismatch: bad/different input data, versus AMIRIS's internal modelling logic genuinely differing from reality.

The new approach removes that ambiguity: the department will supply the same underlying assumptions Energy Brainpool used to generate its purchased price data (capacity figures, fuel prices, etc.) for an agreed year. Those exact assumptions will be fed into AMIRIS, and AMIRIS's resulting price will be compared directly against Brainpool's own price output - not just against real history. If the inputs are identical and the outputs still diverge, the cause is isolated to modelling logic rather than data quality. This is a standard technique in energy-model research, sometimes called a model inter-comparison study, and is a stronger methodology than the original independent-data design.

Why 2019 is the natural candidate year

Germany2019 already has AMIRIS-vs-real-price results from the backtesting phase (Section 9: MAE 5.37, correlation 0.857 - one of the best-fitting years). Running the same year against Brainpool's assumptions would complete a genuine three-way comparison (AMIRIS vs. Brainpool vs. real prices) using infrastructure that already exists, rather than starting from a different year with no prior AMIRIS results to build on.

Immediate deliverable: the data request

The outstanding task is to identify every input AMIRIS requires, so the department can cross-reference it against what their Brainpool data package actually contains. A standalone document (AMIRIS_Data_Request_for_Department.pdf) was produced covering: demand, the conventional plant fleet (capacity, efficiency, cost, emissions, outages per fuel type), the renewable fleet (capacity, generation profile, subsidy scheme per technology), storage, fuel and CO2 prices, and a set of structural questions (cross-border trade assumptions, bidding behaviour, whether negative prices appear in Brainpool's output).

One gap was flagged explicitly rather than assumed away: Brainpool's data most likely includes capacity totals per technology, but may or may not include the hourly weather-driven generation shape AMIRIS also needs. Wherever Brainpool's data does not cover an item, AMIRIS's own independently-sourced data (the same ENTSO-E/SMART pipeline built for the 2023 work) will be used instead - and this substitution will be documented per item, since it is a second, honest candidate explanation for any observed result gap, alongside genuine modelling-logic differences.

Plan once Brainpool data arrives

- Gap analysis: mark each required input as supplied, partially supplied, or backfilled with AMIRIS's own data - this table becomes part of the thesis methodology chapter.
- Build the scenario using Brainpool's assumptions wherever available.
- Run AMIRIS and compare its price against both Brainpool's price and the real 2019 price (three-way comparison).
- If results match closely: evidence supporting AMIRIS as a free substitute once an input pipeline exists. If they diverge: apply the same root-cause methodology already proven on the 2018 findings (Section 10) to identify the specific modelling difference responsible.

17. Resuming This Work in Claude Code

Claude Code automatically saves conversation history to disk, scoped to the project directory. From a terminal opened in the Amiris folder:

- `claude --continue` (or `-c`): reopens the most recent conversation in this directory
- `claude --resume` (or `-r`): opens a picker listing all past sessions in this directory

18. Setting This Up in Claude Desktop

Claude Desktop's Projects feature is a knowledge/context container for chat - it does not, by itself, read local files or run commands. Two separate things were distinguished:

A. Project knowledge (context only, no file access)

- Create a Project named e.g. 'AMIRIS'
- Add custom instructions summarising the local paths, environment, and goal
- Upload small reference files (README.md files, metadata.json, one scenario.yaml) as Project knowledge; skip large timeseries CSVs, output.pb, and schema.yaml

B. Actual file/command access (to replicate hands-on work)

- Requires enabling MCP servers via Settings > Developer > Edit Config
- Official Filesystem MCP server can be scoped to the Amiris folder for file read/write
- No official Anthropic shell-execution MCP server is bundled; running `amiris/python` commands still requires a terminal, or a community MCP server granting command execution (a meaningful security tradeoff to weigh deliberately)

Recommendation given: keep the run-execute-analyze loop in Claude Code (or a terminal) and use Desktop mainly for reviewing results and conceptual discussion, with Project knowledge set up as in (A).